

Operating System

Complete MCQ-Oriented Notes

IBPS SO (IT Officer) Examination - Complete Preparation Series

100% Syllabus Coverage - MCQ-Oriented - Zero Filler

Compiled by Asabhya Maanav

Table of Contents

PART A - OS Fundamentals: Introduction	6
1.1 Definition, Goals and Functions of an OS	6
1.2 OS as Resource Manager and Extended Machine	6
1.3 Types of Operating Systems	6
1.4 Multiprogramming vs Multitasking vs Multiprocessing	7
1.5 OS Structures	7
1.6 Kernel Mode vs User Mode; Dual Mode	8
PART A - System Calls and Services	9
2.1 System Call Concept and Mechanism	9
2.2 User-to-Kernel Mode Transition	9
2.3 Categories of System Calls	9
2.4 Examples: fork, exec, wait, exit	9
2.5 API vs System Call	10
PART B - Processes	11
3.1 Program vs Process	11
3.2 Process Memory Layout	11
3.3 Process States and State-Transition Diagram	11
3.4 Process Control Block (PCB)	11
3.5 Context Switch	12
3.6 Process Creation and Termination	12
3.7 Orphan and Zombie Processes	12
3.8 fork() Output / Process-Count Problems	13
PART B - Threads	14
4.1 Thread vs Process	14
4.2 Benefits of Threads	14
4.3 User-Level vs Kernel-Level Threads	14
4.4 Multithreading Models	14
4.5 Concurrency vs Parallelism	15
PART B - CPU Scheduling	16
5.1 Schedulers: Long-, Short-, Medium-term	16
5.2 Dispatcher and Dispatch Latency	16
5.3 Scheduling Criteria	16
5.4 Preemptive vs Non-preemptive	17

5.5.1 FCFS (First-Come, First-Served)	17
5.5.2 SJF (Shortest Job First, Non-preemptive)	17
5.5.3 SRTF (Shortest Remaining Time First)	17
5.5.4 Priority Scheduling	17
5.5.5 Round Robin (RR)	18
5.5.6 Multilevel Queue (MLQ)	18
5.5.7 Multilevel Feedback Queue (MLFQ)	18
5.6 & 5.7 Gantt Charts and Computing TAT/WT/Response	19
PART C - Inter-Process Communication (IPC)	20
6.1 Shared Memory Model	20
6.2 Message Passing	20
6.3 Pipes, Message Queues, Sockets	20
6.4 Signals	20
PART C - Process Synchronization	22
7.1 Race Condition	22
7.2 Critical Section Problem	22
7.3 Requirements for a Solution	22
7.4 Peterson's Algorithm	22
7.5 Hardware Support: Test-and-Set, Compare-and-Swap	22
7.6 Semaphores	23
7.7 Mutex Locks vs Binary Semaphore	23
7.8 Monitors and Condition Variables	23
7.9.1 Producer-Consumer (Bounded Buffer)	24
7.9.2 Readers-Writers	24
7.9.3 Dining Philosophers	24
7.10 Priority Inversion	24
PART D - Deadlocks	25
8.1 System Model	25
8.2 Four Necessary Conditions	25
8.3 Resource Allocation Graph (RAG)	25
8.4.1 Deadlock Prevention	25
8.4.2 Deadlock Avoidance: Banker's Algorithm	26
8.4.3 Deadlock Detection and Recovery	26
8.5 Starvation vs Deadlock; Livelock	26
PART E - Main Memory Management	28

9.1 Address Binding	28
9.2 Logical vs Physical Address; MMU	28
9.3 Base and Limit Registers; Relocation	28
9.4 Static vs Dynamic Loading/Linking	28
9.5 Swapping	29
9.6 Contiguous Allocation	29
9.7 Paging	29
9.8 Segmentation	31
9.9 Segmentation with Paging	31
PART E - Virtual Memory	32
10.1 Concept and Motivation	32
10.2 Demand Paging; Page Fault Handling	32
10.3 Effective Access Time with Page Faults	32
10.4 Page Replacement Algorithms	32
10.5 Frame Allocation	33
10.6 Thrashing; Locality; Working Set	33
10.7 Page Size Trade-offs	34
PART F - File Systems	35
11.1 File Concept, Attributes, Types, Operations	35
11.2 Access Methods	35
11.3 Directory Structures	35
11.4 File Protection and Access Permissions	35
11.5 File Allocation Methods	36
11.6 Free-Space Management	37
11.7 Directory Implementation	37
PART F - Disk / Secondary Storage	38
12.1 Disk Structure	38
12.2 Disk Performance	38
12.3 Disk Scheduling Algorithms	38
12.4 RAID Levels 0-5	39
PART G - I/O Systems	40
13.1 I/O Hardware	40
13.2 Programmed I/O (Polling)	40
13.3 Interrupt-Driven I/O; ISR	40
13.4 Direct Memory Access (DMA)	40

13.5 Buffering, Caching, Spooling	40
PART G - Protection and Security	42
14.1 Goals of Protection; Access Matrix	42
14.2 Access Control Lists vs Capabilities	42
14.3 Authentication Basics	42
PART H - CIL MT Exam Focus and Numerical Toolkit	43
15.1 CPU Scheduling Formulas (Recap)	43
15.2 Banker's Algorithm (Recap)	43
15.3 Page Replacement (Recap)	43
15.4 EMAT / TLB / Paging (Recap)	43
15.5 Address Translation (Recap)	43
15.6 Disk Scheduling (Recap)	43
15.7 Semaphore Reasoning (Recap)	43
15.8 inode Max File Size (Recap)	43
15.9 fork() Process Count (Recap)	43
15.10 Definitions and One-Liner Anchors	44
Final Rapid-Revision Comparison Table	44

PART A - OS Fundamentals: Introduction

1.1 Definition, Goals and Functions of an OS

- An **Operating System (OS)** is system software that acts as an **intermediary** between the user/application programs and the computer **hardware**, managing all resources and providing a convenient environment for program execution.
- Strict definition: the OS is the **one program running at all times** on the computer, called the **kernel**; everything else is either a **system program** (ships with the OS) or an **application program**.
- Two complementary viewpoints: (1) **top-down** - OS as an **extended/virtual machine** that hides messy hardware details behind clean abstractions; (2) **bottom-up** - OS as a **resource manager** that allocates CPU, memory, and I/O among competing processes.
- Primary **goals**: **convenience** (ease of use), **efficiency** (maximize resource utilization), and **ability to evolve** (allow new features without disruption).
- Core **functions**: **process management**, **memory management**, **file management**, **I/O management**, **secondary-storage management**, **protection and security**, **networking**, and **command interpretation (shell)**.

Memory Trick

- "**PM-FISP-N**" = Process, Memory, File, I/O, Storage, Protection, Networking - the seven management duties.

CBT Trap

- The **kernel** is the core that always runs; the **shell** (command interpreter) is NOT part of the kernel - it is a system program. Examiners swap these.
- "OS as extended machine" emphasizes **abstraction/convenience**; "OS as resource manager" emphasizes **allocation/efficiency**. Do not interchange.

1.2 OS as Resource Manager and Extended Machine

- As an **extended machine**, the OS turns ugly hardware (disk = cylinders, sectors, heads) into clean abstractions (**files, processes, address spaces**).
- As a **resource manager**, it performs **multiplexing** of resources in **two ways**: **time multiplexing** (e.g., CPU shared in turns) and **space multiplexing** (e.g., memory/disk divided among users).
- The OS resolves **conflicts** for resources by enforcing policies for **fairness**, **protection**, and **efficiency**.

CBT Trap

- **Time multiplexing** = sharing in time (CPU, printer queue). **Space multiplexing** = dividing the resource (RAM, disk). MCQs ask which applies to a given resource.

1.3 Types of Operating Systems

- **Batch OS**: jobs with similar needs are **batched** together and run without user interaction; uses **offline** processing and **spooling**; main drawback is **no interaction** and high **turnaround time**.
- **Multiprogramming OS**: keeps **several jobs** in memory at once; when one job waits for I/O, the CPU switches to another, maximizing **CPU utilization**. Goal = keep the CPU **never idle**.
- **Time-sharing / Multitasking OS**: a logical extension of multiprogramming where the CPU switches so fast (via a small **time quantum**) that each user feels they have a **dedicated** machine; optimizes **response time**.
- **Real-Time OS (RTOS)**: produces correct results within a strict **time deadline**. Two kinds: **Hard RTOS** (missing a deadline = catastrophic failure, e.g., **airbags**, pacemakers) and **Soft RTOS** (deadline desirable but not fatal, e.g., **multimedia** streaming).

- **Distributed OS**: independent, networked computers appear to users as a **single** coherent system; benefits = **resource sharing, speedup, reliability**.
- **Network OS**: each machine keeps its own OS and identity; users are **aware** of multiple machines (contrast with distributed which hides them).
- **Multiprocessor OS**: manages systems with **2 or more** CPUs sharing bus/clock/memory; **symmetric (SMP)** = all CPUs are peers; **asymmetric** = one master CPU schedules slaves.

Type	Optimizes	Interaction	Key Idea
Batch	Throughput	None	Group similar jobs
Multiprogramming	CPU utilization	Indirect	Switch on I/O wait
Time-sharing	Response time	Direct	Tiny time quantum
Real-time	Deadline	Direct	Hard vs Soft
Distributed	Resource sharing	Transparent	Many look like one

CBT Trap

- **Multiprogramming** optimizes **CPU utilization**; **time-sharing** optimizes **response time**. A favorite swap question.
- **Hard real-time** = absolute deadline; **soft real-time** = best-effort. Pacemaker/airbag = hard; video playback = soft.

1.4 Multiprogramming vs Multitasking vs Multiprocessing

- **Multiprogramming**: multiple programs reside in memory; CPU switches when current job does I/O. Needs only **one CPU**.
- **Multitasking** (time-sharing): rapid context switching among tasks gives the **illusion** of simultaneity to interactive users; one CPU, driven by a **timer interrupt**.
- **Multiprocessing**: **two or more** physical CPUs execute instructions **truly simultaneously**; provides real **parallelism**.

Concept	CPUs Needed	True Parallelism	Driver
Multiprogramming	1	No	I/O wait
Multitasking	1	No (illusion)	Timer/quantum
Multiprocessing	>=2	Yes	Extra CPUs

CBT Trap

- Only **multiprocessing** gives **true parallelism**; the other two give **concurrency** on a single CPU. Examiners conflate concurrency with parallelism.

1.5 OS Structures

- **Monolithic**: the entire OS runs as a **single program in kernel mode**; fast (no message overhead) but hard to maintain; a bug can crash everything. Example: traditional **UNIX, Linux**.
- **Layered**: OS divided into **layers**, each built on the one below; layer **0** = hardware, top layer = user interface. Easy to debug/verify but **slow** due to traversal and hard to define layer boundaries. Example: **THE** system.
- **Microkernel**: keep only the **minimum** in kernel (IPC, scheduling, basic memory); move the rest (file system, drivers) to **user space** as servers. Benefits = **reliability, security**, easy extension; cost = **message-passing overhead**. Example: **Mach, QNX, L4**.
- **Hybrid**: combines monolithic speed with microkernel modularity (e.g., **Windows NT, macOS**).
- **Modules**: loadable kernel modules (**LKM**) - most modern OSes use this (Linux, Solaris).

Structure	Kernel Size	Speed	Reliability	Example
Monolithic	Large	Fast	Low	Linux
Layered	Medium	Slow	Medium	THE
Microkernel	Minimal	Slow (IPC)	High	QNX, Mach
Hybrid	Medium	Fast	Medium-High	Windows NT

CBT Trap

- **Microkernel** has the **smallest** kernel and uses **message passing**; it is more **reliable** but **slower**. Do not call it faster.

1.6 Kernel Mode vs User Mode; Dual Mode

- **Dual-mode** operation uses a **mode bit** in hardware: **0 = kernel (monitor/supervisor) mode**, **1 = user mode**.
- **Privileged instructions** (I/O, halt, set timer, change mode bit, modify memory-management registers) execute **only** in **kernel mode**; attempting them in user mode raises a **trap**.
- A **system call** switches mode from **user to kernel**; on return, mode flips back to **user**.
- Purpose: protect the OS and other processes from **buggy or malicious** user programs.
- Some CPUs offer **more than 2** modes (e.g., x86 **rings 0-3**; ring 0 = kernel, ring 3 = user).

Memory Trick

- "**0 = OS**, **1 = Owner(user)**" for the mode bit value.

CBT Trap

- The mode bit value **0 = kernel** and **1 = user** is standard textbook convention (Silberschatz). A trap/interrupt sets mode to **kernel** automatically.
- **Privileged instruction** attempted in user mode -> **trap to OS**, not silent ignore.

PART A - System Calls and Services

2.1 System Call Concept and Mechanism

- A **system call** is the programmatic way a **user program requests a service** from the OS kernel; it is the **only** legitimate entry point into the kernel for user code.
- Mechanism: the program executes a special **trap/software interrupt** instruction (e.g., **int 0x80** historically, **syscall** on x86-64) that switches the CPU to **kernel mode** and jumps to a fixed **handler** via the **interrupt/trap vector**.
- A **system-call number** identifies which service is wanted; parameters are passed via **registers**, a **block/table in memory** (address in a register), or the **stack**.

2.2 User-to-Kernel Mode Transition

- Steps: (1) user program loads syscall number + args, (2) executes **trap** instruction, (3) hardware switches to **kernel mode** and saves state, (4) kernel **dispatcher** indexes the **system-call table**, (5) the service runs, (6) result returned and mode flips back to **user**.
- This controlled transition is the foundation of **protection**: user code can never directly run privileged operations.

2.3 Categories of System Calls

- **Process control**: **fork**, **exec**, **wait**, **exit**, **abort**, load, end.
- **File management**: **open**, **read**, **write**, **close**, create, delete, **lseek**.
- **Device management**: request, release, **read**, **write**, **ioctl**.
- **Information maintenance**: **getpid**, **alarm**, **sleep**, get/set time, get/set system data.
- **Communications**: **pipe**, **shmget**, **send**, **recv**, create/delete connection.
- **Protection**: **chmod**, **umask**, **chown**, set permissions.

Category	Representative Calls
Process control	fork , exec , wait , exit
File management	open , read , write , close
Device management	ioctl , read , write
Information	getpid , alarm , sleep
Communication	pipe , shmget , send , recv
Protection	chmod , chown , umask

2.4 Examples: fork, exec, wait, exit

- **fork()**: creates a **new child process** that is a **copy** of the parent; returns **0 to the child** and the **child's PID to the parent**, and **-1** on failure.
- **exec()** family (execvp, execvp): **replaces** the current process image with a **new program**; on success it **never returns**.
- **wait()**: parent **blocks** until a child terminates and collects its **exit status**, preventing **zombies**.
- **exit()**: terminates the calling process and returns a **status code** to the parent.

CBT Trap

- **fork** returns **twice** (once in each process); **exec** returns **zero times** on success. This double/zero return is a classic MCQ.
- A successful **exec** keeps the **same PID** - it does not create a new process.

2.5 API vs System Call

- An **API (Application Programming Interface)** is a set of functions a programmer calls (e.g., **POSIX**, **Win32**, Java API); the underlying **system call** is the actual kernel entry.
- Programmers prefer APIs for **portability** and simplicity; the **run-time support library** maps API calls to system calls.

CBT Trap

- **printf()** is a **library/API** function; it internally invokes the **write()** system call. Examiners ask "which is a system call" - choose **write**, not **printf**.

PART B - Processes

3.1 Program vs Process

- A **program** is a **passive** entity - an executable file stored on disk (a set of instructions).
- A **process** is an **active** entity - a program **in execution**, with a program counter, registers, stack, and resources.
- One program can spawn **many** processes (e.g., multiple instances of a browser); a process is a program + its **execution context**.

Memory Trick

- "Program = recipe (passive); Process = cooking (active)."

3.2 Process Memory Layout

- A process address space has **four** main segments: **text**, **data**, **heap**, and **stack**.
- **Text**: the compiled **program code** (read-only, shareable).
- **Data**: **global** and **static** variables; split into **initialized** data and **BSS** (uninitialized).
- **Heap**: **dynamically** allocated memory (**malloc/new**); **grows upward** (toward higher addresses).
- **Stack**: **function call frames**, local variables, return addresses; **grows downward** (toward lower addresses).
- The heap and stack grow **toward each other**; collision = **stack overflow**/out of memory.

Segment	Contents	Growth
Text	Code	Fixed
Data	Globals/statics	Fixed
Heap	malloc/new	Upward
Stack	Locals/frames	Downward

CBT Trap

- **Heap grows up, stack grows down** (in conventional layout). Local variables -> **stack**; dynamic memory -> **heap**; global/static -> **data**.

3.3 Process States and State-Transition Diagram

- The **five** classic states are: **new**, **ready**, **running**, **waiting (blocked)**, and **terminated**.
- **new** -> **ready**: admitted by the long-term scheduler.
- **ready** -> **running**: **dispatch** by the short-term scheduler (dispatcher).
- **running** -> **ready**: **interrupt**/time-quantum expiry (preemption).
- **running** -> **waiting**: issues an **I/O** or **wait** request.
- **waiting** -> **ready**: **I/O completion**/event occurs.
- **running** -> **terminated**: **exit**.
- Only **one** process is **running** per CPU at a time; many can be **ready** or **waiting**.

CBT Trap

- There is **no** direct **waiting** -> **running** transition; a blocked process must go to **ready** first. Frequently tested.
- **running** -> **ready** is caused by **preemption** (timer), NOT by I/O; **running** -> **waiting** is caused by I/O.

3.4 Process Control Block (PCB)

- The **PCB** (a.k.a. **Task Control Block**) is the kernel data structure that stores all information about a process.
- Contents: **process state**, **process ID (PID)**, **program counter**, **CPU registers**, **CPU-scheduling info** (priority, queue pointers), **memory-management info** (base/limit, page tables), **accounting info** (CPU used, time limits), and **I/O status** (open files, devices).
- The PCB is the **repository** that makes **context switching** possible.

Memory Trick

- **"PC-PRIMA"**: PC (program counter), PID, Registers, I/O status, Memory info, Accounting + state.

CBT Trap

- The **program counter** and **CPU registers** are saved in the **PCB** during a context switch - not on the user stack. Examiners ask where saved state lives.

3.5 Context Switch

- A **context switch** is the act of saving the state of the current process into its **PCB** and loading the saved state of the next scheduled process.
- It is **pure overhead** - no useful work is done during the switch; the system does **nothing useful** while switching.
- Cost depends on hardware (number of registers, presence of multiple register sets); typical times are a few **microseconds**.
- Triggered by **interrupts**, **system calls**, or **scheduling** decisions.

CBT Trap

- Context-switch time is **overhead**; it does **not** count as throughput. Smaller **time quantum** -> **more** context switches -> **more** overhead.

3.6 Process Creation and Termination

- **Creation**: parent uses **fork()** to create a child; child may then **exec()** a new program. Processes form a **tree**; the root in UNIX is **init/systemd (PID 1)**.
- After fork, parent and child can run **concurrently**; the parent may **wait()** or run in parallel.
- **Termination**: a process ends via **exit()** (normal) or is killed via **abort/kill** (abnormal). The parent collects status with **wait()**.
- **Cascading termination**: in some systems, terminating a parent forces termination of all **children**.

CBT Trap

- The **first** user process in UNIX has **PID 1** (**init/systemd**) and adopts all **orphans**.

3.7 Orphan and Zombie Processes

- **Zombie** (defunct): a process that has **terminated** but whose **exit status** has not yet been read by the parent via **wait()**; it occupies only a **PCB entry**, no other resources.
- **Orphan**: a process whose **parent terminated first**; it is **re-parented (adopted)** by **init/systemd (PID 1)**, which periodically reaps it.

Aspect	Zombie	Orphan
Who died	Child	Parent
Child state	Terminated, not reaped	Still running
Fix/Adopter	Parent calls wait()	Adopted by init (PID 1)

CBT Trap

- **Zombie** = child dead, parent alive but not waiting. **Orphan** = parent dead, child alive. Reversing these is the #1 trap here.

3.8 fork() Output / Process-Count Problems

- After **n** consecutive **fork()** calls, the total number of processes is **2^n** , so the number of **new child** processes created is **$2^n - 1$** .
- Reasoning: each fork **doubles** the number of processes because both parent and child continue from the statement after fork.
- For **k** forks in a loop / sequence with no conditionals, child count = **$2^k - 1$** .
- A common variant: `fork(); fork(); fork();` -> **8** total processes, **7** children.

Memory Trick

- "n forks -> **2^n** total, **$2^n - 1$** children." Count the **number of fork statements actually executed**.

CBT Trap

- If forks are joined by **&&/||** or guarded by **if**, the doubling does NOT fully apply - trace each branch. Print statements after fork run **once per existing process**.

PART B - Threads

4.1 Thread vs Process

- A **thread** is the **basic unit of CPU utilization** - a lightweight flow of control within a process, comprising a **thread ID**, **program counter**, **register set**, and **stack**.
- Threads of the same process **share**: **code/text**, **data**, **heap**, and **open files**; they have **separate**: **stack**, **registers**, **PC**, and **thread ID**.
- Processes are **heavyweight** (separate address spaces); threads are **lightweight** (shared address space).

Resource	Shared by threads?
Code/Text	Yes
Global data / Heap	Yes
Open files	Yes
Stack	No (private)
Registers / PC	No (private)

CBT Trap

- Threads share **heap and global data** but each has its **own stack and registers**. "Threads share the stack" is FALSE.

4.2 Benefits of Threads

- Four** classic benefits: **responsiveness**, **resource sharing**, **economy** (cheaper than process creation/context switch), and **scalability** (parallelism on multicore).
- Thread creation/switching is **faster** than process creation/switching because the **address space** is not changed.

4.3 User-Level vs Kernel-Level Threads

- User-level threads (ULT)**: managed by a **user library** (e.g., POSIX pthreads in user space); the kernel is **unaware**. Fast switching, but a **single blocking system call** blocks the **whole process**.
- Kernel-level threads (KLT)**: managed directly by the **kernel**; the kernel schedules threads, so one blocking thread does **not** block others; switching is **slower** (needs kernel involvement).

Feature	User-level	Kernel-level
Managed by	Library	Kernel
Switch speed	Fast	Slower
Blocking call	Blocks whole process	Blocks one thread
True parallelism	No	Yes

CBT Trap

- In pure **user-level** threading, one thread's **blocking system call** blocks all threads of that process. KLT avoids this. Classic exam contrast.

4.4 Multithreading Models

- Many-to-One**: many ULTs map to **one** KLT; no true parallelism; one block stops all. Example: Green threads.

- **One-to-One**: each ULT maps to a **distinct** KLT; true **parallelism** but per-thread kernel overhead limits count. Example: **Windows, Linux**.
- **Many-to-Many**: **m** ULTs multiplexed onto **n** KLTs ($n \leq m$); best of both - parallelism without unlimited kernel threads.
- **Two-level model**: many-to-many plus the ability to **bind** a user thread to a kernel thread.

CBT Trap

- **One-to-one** gives the most concurrency but the most **overhead**; **many-to-one** gives least overhead but **no parallelism**. Match model to property carefully.

4.5 Concurrency vs Parallelism

- **Concurrency**: multiple tasks make progress in **overlapping** time periods; achievable on a **single** core via interleaving.
- **Parallelism**: multiple tasks execute **literally at the same instant**; requires **multiple** cores/CPUs.
- "Concurrency is about **dealing with** many things; parallelism is about **doing** many things at once."

CBT Trap

- **All parallel execution is concurrent**, but not all concurrency is parallel. Single-core systems can be concurrent but **never** truly parallel.

PART B - CPU Scheduling

5.1 Schedulers: Long-, Short-, Medium-term

- **Long-term scheduler (job scheduler)**: selects jobs from the **job pool** (disk) and loads them into **memory** (ready queue); controls the **degree of multiprogramming**; runs **infrequently** (seconds/minutes).
- **Short-term scheduler (CPU scheduler)**: selects which **ready** process gets the **CPU** next; runs very **frequently** (milliseconds); must be **fast**.
- **Medium-term scheduler**: performs **swapping** - removes (swaps out) processes from memory to reduce multiprogramming and later swaps them back in.

Scheduler	Frequency	Controls	Speed Need
Long-term	Rare	Degree of multiprogramming	Slow OK
Short-term	Very often	CPU allocation	Fast
Medium-term	Medium	Swapping	Medium

CBT Trap

- **Long-term** controls the degree of multiprogramming; **short-term** runs most frequently; **medium-term** does swapping. Time-sharing systems often have **no** long-term scheduler.

5.2 Dispatcher and Dispatch Latency

- The **dispatcher** is the module that gives CPU control to the process chosen by the short-term scheduler; it performs **context switch**, switch to **user mode**, and **jump** to the proper location to restart the program.
- **Dispatch latency** = the time the dispatcher takes to **stop one process and start another**; must be **minimal**.

CBT Trap

- The **scheduler** decides **who** runs (policy); the **dispatcher** actually **hands over the CPU** (mechanism). Do not confuse the two.

5.3 Scheduling Criteria

- **CPU utilization**: keep CPU as **busy** as possible (maximize).
- **Throughput**: number of processes **completed per unit time** (maximize).
- **Turnaround time (TAT)**: **completion time - arrival time** = total time from submission to completion (minimize).
- **Waiting time (WT)**: **turnaround time - burst time** = time spent in the **ready queue** (minimize).
- **Response time**: time from submission until the **first response**/first CPU allocation (minimize) - key for interactive systems.

Criterion	Goal	Formula
CPU utilization	Max	busy/total
Throughput	Max	jobs/time
Turnaround	Min	CT - AT
Waiting	Min	TAT - BT
Response	Min	first CPU - AT

Memory Trick

- "**TAT = CT - AT**", "**WT = TAT - BT**", "**Response = First start - AT**". Memorize these three.

CBT Trap

- **Waiting time = TAT - Burst** (NOT TAT - response). **Response time** counts only until the **first** CPU allocation, while waiting time is total ready-queue time.

5.4 Preemptive vs Non-preemptive

- **Non-preemptive**: once the CPU is given to a process, it keeps it until it **terminates** or **switches to waiting** (voluntary release). Examples: **FCFS**, **SJF**, non-preemptive priority.
- **Preemptive**: the OS can **forcibly** take the CPU away (timer, higher-priority arrival). Examples: **Round Robin**, **SRTF**, preemptive priority.
- Preemption can cause **race conditions** on shared data and needs synchronization.

CBT Trap

- **FCFS** and **SJF** (basic) are **non-preemptive**; **SRTF** and **RR** are **preemptive**. SJF preemptive = **SRTF**.

5.5.1 FCFS (First-Come, First-Served)

- **FCFS** schedules processes in **arrival order** using a **FIFO queue**; **non-preemptive**.
- Simple and **fair** in arrival sense, but suffers the **convoy effect**: short processes wait behind one long process, raising average waiting time.
- Average waiting time is generally **not minimal** and varies with arrival order.

CBT Trap

- **Convoy effect** is the signature drawback of **FCFS**. FCFS can give very high **average waiting time** if a long job arrives first.

5.5.2 SJF (Shortest Job First, Non-preemptive)

- **SJF** selects the ready process with the **smallest CPU burst**; **non-preemptive**.
- It is **provably optimal**: gives the **minimum average waiting time** for a given set of processes.
- Drawback: requires **knowing/predicting** the next burst length and can cause **starvation** of long jobs.
- Burst prediction uses **exponential averaging**: $\tau(n+1) = \alpha * t(n) + (1 - \alpha) * \tau(n)$, with $0 \leq \alpha \leq 1$.

CBT Trap

- **SJF** gives the **minimum average waiting time** - this is its defining theoretical property. The catch is burst length must be **known/estimated**.

5.5.3 SRTF (Shortest Remaining Time First)

- **SRTF** is the **preemptive** version of SJF: if a newly arrived process has a **shorter remaining burst** than the running one, it **preempts**.
- Gives even **lower** average waiting time than SJF in the presence of varied arrival times, but more **context switches** and worse **starvation**.

CBT Trap

- On each **new arrival**, re-compare **remaining** times. Students forget to re-evaluate at arrival instants - leading to wrong Gantt charts.

5.5.4 Priority Scheduling

- Each process has a **priority**; CPU goes to the **highest priority** (commonly **smallest number = highest priority**).
- Can be **preemptive** or **non-preemptive**.
- Main problem: **starvation/indefinite blocking** of low-priority processes.
- Solution: **aging** - gradually **increase** the priority of processes that wait long.

CBT Trap

- Convention is usually **lower number = higher priority** (but always read the question). **Aging** solves **starvation**, not deadlock.

5.5.5 Round Robin (RR)

- **RR** = FCFS with **preemption** driven by a fixed **time quantum (TQ)**; designed for **time-sharing**.
- If TQ is **very large**, RR degenerates to **FCFS**; if TQ is **very small**, overhead from **context switching** dominates.
- Provides good **response time** and is **starvation-free**.
- Rule of thumb: **80%** of CPU bursts should be shorter than the time quantum.

CBT Trap

- **Large TQ -> FCFS**; **small TQ -> high context-switch overhead**. Number of context switches rises as TQ shrinks. RR is **never** non-preemptive.

5.5.6 Multilevel Queue (MLQ)

- **MLQ** partitions the ready queue into **multiple separate queues** (e.g., **system**, **interactive**, **batch**); each has its **own scheduling algorithm**.
- A process is **permanently** assigned to one queue (**no movement** between queues).
- Scheduling between queues is usually **fixed-priority preemptive** or **time-slice** among queues.

CBT Trap

- In MLQ, processes **do not move** between queues; that movement is the feature of **MLFQ**. Key distinguishing point.

5.5.7 Multilevel Feedback Queue (MLFQ)

- **MLFQ** allows processes to **move between queues** based on their **CPU-burst behavior** (feedback).
- A process using too much CPU is **demoted** to a lower-priority queue; one waiting too long can be **promoted (aging)**.
- Most **general** and configurable scheme; parameters: number of queues, scheduling per queue, demotion/promotion rules.

Feature	MLQ	MLFQ
Queue movement	No	Yes (feedback)
Flexibility	Low	High
Starvation handling	Poor	Via aging

CBT Trap

- **MLFQ** = queues + **feedback (movement)** + aging; it is the most **flexible** and **complex**. MLQ has no movement.

5.6 & 5.7 Gantt Charts and Computing TAT/WT/Response

- A **Gantt chart** is a bar timeline showing which process holds the CPU over time; build it strictly by the algorithm rules.
- Steps: (1) order/insert by **arrival** and rule, (2) draw the timeline, (3) read each process's **completion time (CT)**, (4) compute **TAT = CT - AT**, (5) **WT = TAT - BT**, (6) average over n processes.
- **Average WT** = (sum of WT) / n; **Average TAT** = (sum of TAT) / n.

Process	AT	BT	CT	TAT=CT-AT	WT=TAT-BT
P1	0	5	5	5	0
P2	1	3	8	7	4
P3	2	1	9	7	6

CBT Trap

- Always subtract **arrival time**: **TAT = CT - AT**, not just CT. When arrival times are all **0**, **TAT = CT**, which lulls students into forgetting AT later.

PART C - Inter-Process Communication (IPC)

6.1 Shared Memory Model

- In **shared memory** IPC, cooperating processes establish a **shared region** of memory; one writes, the other reads.
- It is the **fastest** IPC because after setup, communication occurs at **memory speed** with **no kernel involvement** per exchange.
- The **OS** only sets up the region; **synchronization** (avoiding race conditions) is the **programmer's** responsibility.

6.2 Message Passing

- **Message passing** exchanges data via **send()** and **receive()** kernel calls; useful in **distributed** systems and when no shared address space exists.
- **Direct** communication names the **recipient explicitly**; **indirect** uses **mailboxes/ports**.
- **Synchronous (blocking)**: send/receive block until the message is delivered/received; **asynchronous (non-blocking)**: caller continues immediately.
- **Rendezvous**: both blocking send and blocking receive.

Aspect	Shared Memory	Message Passing
Speed	Fast	Slower
Kernel per exchange	No	Yes
Best for	Same machine	Distributed
Synchronization	Programmer	Built-in (often)

CBT Trap

- **Shared memory** is faster but needs manual synchronization; **message passing** is slower but easier and works across machines. Examiners ask "fastest IPC" = **shared memory**.

6.3 Pipes, Message Queues, Sockets

- **Ordinary (anonymous) pipe**: **unidirectional**, byte-stream, used between **related** (parent-child) processes; created via **pipe()**.
- **Named pipe (FIFO)**: persists as a **special file**; **bidirectional** usage possible; works between **unrelated** processes; created via **mkfifo**.
- **Message queue**: kernel-maintained linked list of messages with **types/priorities**.
- **Sockets**: endpoints for communication across a **network** (IP + port); the basis of client-server.

CBT Trap

- **Ordinary pipe** = related processes, unidirectional; **named pipe (FIFO)** = unrelated processes, persists in filesystem. This pair is a classic distinction.

6.4 Signals

- A **signal** is an **asynchronous notification** sent to a process to notify it of an event (e.g., **SIGINT** from Ctrl+C, **SIGKILL (9)**, **SIGSEGV**, **SIGTERM (15)**).
- A process can **catch**, **ignore**, or use the **default** action; **SIGKILL (9)** and **SIGSTOP** cannot be caught or ignored.

CBT Trap

- **SIGKILL (signal 9)** and **SIGSTOP** are the **only** signals that **cannot** be caught, blocked, or ignored. A favorite Linux MCQ.

PART C - Process Synchronization

7.1 Race Condition

- A **race condition** occurs when **multiple processes/threads** access **shared data** concurrently and the **final result depends on the timing/order** of execution.
- Caused by **unsynchronized** access to a shared resource during a **non-atomic** update (e.g., `count++` which is really **load, increment, store**).

CBT Trap

- A race condition arises only with **shared writable data** + **concurrent access** + **non-atomic** operation. `count++` and `count--` interleaving is the textbook example.

7.2 Critical Section Problem

- The **critical section (CS)** is the segment of code where a process **accesses shared resources**; only **one** process may be in its CS at a time.
- Structure: **entry section** -> **critical section** -> **exit section** -> **remainder section**.

7.3 Requirements for a Solution

- **Three** requirements: **mutual exclusion** (at most one in CS), **progress** (if no one is in CS, selection of next entrant cannot be postponed indefinitely), and **bounded waiting** (a bound on how many times others enter before a waiting process gets in).

Memory Trick

- "**MPB** = Mutual exclusion, Progress, Bounded waiting."

CBT Trap

- **Progress** (no needless blocking) and **Bounded waiting** (no starvation) are distinct. A solution can give mutual exclusion yet fail progress or bounded waiting.

7.4 Peterson's Algorithm

- **Peterson's solution** is a classic **software** solution for **two** processes using two shared variables: a boolean array **flag[2]** and an int **turn**.
- Entry: set own **flag[i] = true**, set **turn = j**, then **busy-wait** while **flag[j] && turn == j**.
- It satisfies all **three** requirements (**mutual exclusion, progress, bounded waiting**) but assumes **atomic** loads/stores and no instruction reordering.

CBT Trap

- Peterson's algorithm works for **exactly 2** processes and relies on **turn + flag[]**; it can fail on modern CPUs due to **instruction reordering** (needs memory barriers). It uses **busy waiting (spinlock)**.

7.5 Hardware Support: Test-and-Set, Compare-and-Swap

- **Test-and-Set (TAS)**: an **atomic** instruction that returns the old value of a lock and sets it to **true** in one indivisible step.
- **Compare-and-Swap (CAS)**: atomically compares a variable to an **expected** value and, if equal, sets it to a **new** value; returns the original.

- These provide **atomicity** in hardware, enabling correct locks without disabling interrupts on multiprocessors.

CBT Trap

- **TAS** and **CAS** are **atomic hardware instructions**; they enable mutual exclusion but naive use still has **busy waiting** and may violate **bounded waiting** unless extended.

7.6 Semaphores

- A **semaphore** is an integer variable accessed only through two **atomic** operations: **wait() / P() / down()** and **signal() / V() / up()**.
- **wait(S)**: while (S <= 0); S--; (decrement, block if not available).
- **signal(S)**: S++; (increment, possibly wake a waiter).
- **Counting semaphore**: integer can range over an **unrestricted domain**; used to control access to a resource with **N** instances.
- **Binary semaphore**: value restricted to **0 or 1**; behaves like a **mutex lock**.
- To avoid busy waiting, implement with a **waiting queue + block()/wakeup()**.

Type	Range	Use
Counting	0..N	N identical resources
Binary	0 or 1	Mutual exclusion

Memory Trick

- "**P = Proberen (try, decrement)**, **V = Verhogen (increase)**" - Dijkstra's Dutch terms.

CBT Trap

- **wait = P = down = decrement**; **signal = V = up = increment**. Counting semaphore initialized to **N** = number of available instances. Initialized to **1** = mutex.

7.7 Mutex Locks vs Binary Semaphore

- A **mutex** enforces mutual exclusion and has the concept of **ownership**: only the thread that **locked** it may **unlock** it.
- A **binary semaphore** has **no ownership**: any thread can **signal** it; it can be used for **signaling/ordering** between threads.

Feature	Mutex	Binary Semaphore
Ownership	Yes	No
Purpose	Locking	Locking + signaling
Who releases	Locker only	Anyone

CBT Trap

- **Mutex has ownership**; binary semaphore does **not**. Use a **semaphore** (not mutex) for **signaling/ordering** between two different threads.

7.8 Monitors and Condition Variables

- A **monitor** is a high-level synchronization construct: an **abstract data type** where only **one** process can be active inside at a time (**automatic mutual exclusion**).
- **Condition variables** inside a monitor support **wait()** (suspend) and **signal()** (resume one waiter); a monitor's **signal** differs from a semaphore's - if no one is waiting, the signal is **lost (no effect)**.

CBT Trap

- Monitor **signal** on an empty condition variable is **lost**; semaphore **signal** always **increments** the count (remembered). This is a key monitor-vs-semaphore distinction.

7.9.1 Producer-Consumer (Bounded Buffer)

- Uses **three** semaphores: **mutex** (init **1**) for buffer access, **empty** (init **n**) counting empty slots, and **full** (init **0**) counting filled slots.
- Producer: **wait(empty)**, **wait(mutex)**, add, **signal(mutex)**, **signal(full)**.
- Consumer: **wait(full)**, **wait(mutex)**, remove, **signal(mutex)**, **signal(empty)**.

CBT Trap

- **empty** starts at buffer size **n**; **full** starts at **0**; **mutex** at **1**. Reversing wait order (mutex before empty/full) causes **deadlock**. The resource semaphore must be acquired **before** mutex.

7.9.2 Readers-Writers

- **Multiple readers** may read concurrently, but a **writer** needs **exclusive** access.
- First (reader-priority) solution uses **read_count**, a **mutex** protecting it, and a **wrt/rw** semaphore; can **starve writers**.
- Variants give **writer priority** or **fairness**.

CBT Trap

- The basic readers-writers solution can cause **writer starvation**. Readers entering keep the write-lock held continuously.

7.9.3 Dining Philosophers

- **5** philosophers, **5** chopsticks; each needs **2 adjacent** chopsticks to eat; naive "pick left then right" can **deadlock** (circular wait).
- Solutions: allow at most **4** to sit, pick up both chopsticks **atomically**, or make **odd/even** philosophers pick in **opposite order (asymmetric)**.

CBT Trap

- The classic deadlock arises when **all 5** grab their **left** chopstick simultaneously -> **circular wait**. The fix "at most **n-1** = 4 philosophers eat" breaks it.

7.10 Priority Inversion

- **Priority inversion**: a **high-priority** task is blocked waiting for a resource held by a **low-priority** task, while a **medium-priority** task preempts the low one - effectively inverting priorities.
- Remedy: **priority inheritance** - the low-priority holder temporarily **inherits** the high task's priority.

CBT Trap

- The famous **Mars Pathfinder** bug was **priority inversion**, solved by **priority inheritance**. Do not confuse with deadlock or starvation.

PART D - Deadlocks

8.1 System Model

- Resources are of types **R1, R2, ...**, each with one or more **instances**; a process must **request**, **use**, then **release** a resource.
- A **deadlock** is a set of processes in which **each is waiting** for an event (resource release) that **only another waiting process can cause** - so none ever proceeds.

8.2 Four Necessary Conditions

- All four** must hold **simultaneously** for deadlock: **mutual exclusion**, **hold and wait**, **no preemption**, and **circular wait**.
- Mutual exclusion**: at least one resource is **non-sharable**.
- Hold and wait**: a process **holds** at least one resource while **waiting** for others.
- No preemption**: resources cannot be **forcibly** taken; released only **voluntarily**.
- Circular wait**: a **closed chain** of processes each waiting for a resource held by the next.

Memory Trick

- "**MHN-C**: Mutual exclusion, Hold-and-wait, No-preemption, Circular wait." All **4** needed together.

CBT Trap

- All four are **necessary but not sufficient** individually; with **single-instance** resources a **cycle implies deadlock**, but with **multiple instances** a cycle is necessary, **not sufficient**.

8.3 Resource Allocation Graph (RAG)

- A **RAG** has process nodes (circles) and resource nodes (rectangles with dots for instances); **request edge** P → R, **assignment edge** R → P.
- Single-instance** resources: a **cycle** ↔ **deadlock**.
- Multiple-instance** resources: a cycle is **necessary** but **not sufficient** - may or may not be a deadlock.

CBT Trap

- No cycle => no deadlock** (always). **Cycle + single instance => deadlock**. **Cycle + multi-instance => maybe**. Memorize all three cases.

8.4.1 Deadlock Prevention

- Prevent deadlock by **ensuring at least one** of the four conditions **cannot hold**.
- Negate **mutual exclusion**: make resources **sharable** (not always possible).
- Negate **hold and wait**: require a process to request **all** resources at once, or release all before requesting more; causes **low utilization**/starvation.
- Negate **no preemption**: allow **preemption** - if a process holding resources requests one that cannot be granted, it must **release** all held.
- Negate **circular wait**: impose a **total ordering** of resource types and require requests in **increasing** order.

CBT Trap

- Circular wait** is negated by **ordering resources** (request in increasing index). **Hold-and-wait** negation = request all at once. Map each negation to the right condition.

8.4.2 Deadlock Avoidance: Banker's Algorithm

- Avoidance** uses **a priori** information about **maximum** resource needs to keep the system in a **safe state**.
- A state is **safe** if there exists a **safe sequence** - an ordering of all processes such that each can obtain its maximum needs from currently **available + released** resources.
- Safe => no deadlock; unsafe => possibility (not certainty)** of deadlock.
- Data structures: **Available[m]**, **Max[n][m]**, **Allocation[n][m]**, **Need[n][m]**, where **Need = Max - Allocation**.
- Safety algorithm**: find a process whose **Need <= Work**; if found, pretend it finishes, add its Allocation to **Work**, mark **Finish=true**; repeat. If all finish, state is **safe**.
- Resource-Request algorithm**: when P_i requests, check **Request <= Need** and **Request <= Available**, tentatively allocate, then run the **safety check**; grant only if state stays **safe**.

Matrix	Meaning
Available	Free instances per resource type
Max	Max demand of each process
Allocation	Currently held
Need	Max - Allocation

Memory Trick

- "**Need = Max - Allocation**." Safety = find a process with **Need <= Work**, free it, repeat.

CBT Trap

- Unsafe state is NOT the same as deadlock** - it only means deadlock is possible. Banker's algorithm is for **multiple instances**; the single-instance avoidance method is the **resource-allocation-graph (claim edge)** algorithm.

8.4.3 Deadlock Detection and Recovery

- Allow deadlocks, then **detect** them with a **wait-for graph** (single instance) or a **detection algorithm** (multi-instance, like the safety algorithm without Max).
- Recovery**: (a) **process termination** - abort all deadlocked processes or abort one at a time; (b) **resource preemption** - select a **victim**, **rollback**, and prevent **starvation**.

CBT Trap

- A **wait-for graph** is obtained by **removing resource nodes** from the RAG (single-instance only); a **cycle** there means deadlock. Detection differs from avoidance: detection acts **after**, avoidance acts **before**.

8.5 Starvation vs Deadlock; Livelock

- Deadlock**: processes are **blocked forever** in a circular wait; **no state change**.
- Starvation (indefinite blocking)**: a process waits indefinitely because others are **always preferred**; the process is **not blocked in a cycle**.
- Livelock**: processes **keep changing state** in response to each other but make **no progress** (they are not blocked but stuck).

Term	Blocked?	State Changes?
Deadlock	Yes	No
Starvation	Maybe	Yes (others run)
Livelock	No	Yes (but no progress)

CBT Trap

- **Livelock** != deadlock: in livelock processes are **active** (changing state) but achieve nothing; in deadlock they are **stuck/blocked**.

PART E - Main Memory Management

9.1 Address Binding

- **Address binding** maps program addresses to physical memory and can happen at **three** times: **compile time**, **load time**, or **execution time**.
- **Compile-time** binding: if memory location is known in advance, generate **absolute code**; must recompile if start address changes.
- **Load-time** binding: compiler generates **relocatable code**; final binding done at load.
- **Execution-time** binding: binding deferred to **run time**; needed if a process can be **moved** during execution; requires **hardware (MMU)**.

CBT Trap

- **Execution-time** binding is the most **flexible** and is required for **swapping**/relocation; it needs **MMU** hardware support. Compile-time = absolute code.

9.2 Logical vs Physical Address; MMU

- **Logical (virtual) address**: generated by the **CPU**; the program's view.
- **Physical address**: the address seen by the **memory unit** (RAM).
- The **Memory Management Unit (MMU)** translates logical to physical addresses at **run time**.
- The set of all logical addresses is the **logical address space**; physical addresses form the **physical address space**.

CBT Trap

- **Compile-time** and **load-time** binding produce **identical** logical and physical addresses; only **execution-time** binding makes them **differ**. The **user program** deals only with **logical** addresses.

9.3 Base and Limit Registers; Relocation

- The **base (relocation) register** holds the **smallest legal physical address**; the **limit register** holds the **size/range**.
- For every logical address, the MMU adds the **base** and checks it is **< limit** (else a **trap**/addressing error).
- Physical address = **logical address + base register** (in the simple relocation scheme).

CBT Trap

- The **limit register** bounds the **size**, not the top address; an access with logical address **>= limit** causes a **trap to OS**. Base+limit gives both relocation and **protection**.

9.4 Static vs Dynamic Loading/Linking

- **Static loading**: the entire program is loaded into memory before execution.
- **Dynamic loading**: a routine is loaded **only when called** -> better memory utilization for rarely-used code.
- **Static linking**: libraries are **combined into the binary** at link time (larger executable).
- **Dynamic linking**: linking deferred to **run time** via **stubs**; shared libraries (**DLL/.so**) are loaded once and **shared**.

CBT Trap

- **Dynamic linking** needs OS support (to check if the library is already in memory) and enables **shared libraries**; **dynamic loading** does **not** need special OS support. Distinguish loading vs linking.

9.5 Swapping

- **Swapping** temporarily moves a process from **memory to a backing store (disk)** and back, to allow the total memory of all processes to **exceed** physical memory.
- The **medium-term scheduler** performs swapping; **swap time** is dominated by **transfer time** proportional to memory size.

CBT Trap

- Swapping's major cost is **transfer time** (proportional to data moved), not seek time; reducing swapped data reduces swap time. **Standard swapping** is rare in modern OSes (replaced by paging-based swapping).

9.6 Contiguous Allocation

- In **contiguous allocation**, each process occupies a **single contiguous** block of memory.

9.6.1 Fixed Partitioning

- Memory divided into **fixed-size** partitions; degree of multiprogramming bounded by **number of partitions**.
- Suffers **internal fragmentation**: allocated block is **larger** than requested; the leftover **inside** the partition is wasted.

9.6.2 Variable Partitioning

- Partitions created **dynamically** to fit each process; no internal fragmentation but suffers **external fragmentation**: free memory exists but is **scattered** in small non-contiguous holes.

9.6.3 Placement Algorithms

- **First-fit**: allocate the **first** hole big enough; **fastest**.
- **Best-fit**: allocate the **smallest** hole that fits; leaves **tiny** leftover holes; must search all (slow); often worst for fragmentation.
- **Worst-fit**: allocate the **largest** hole; leaves a large usable remainder but consumes big holes.

9.6.4 Compaction

- **Compaction** shuffles allocated blocks together to merge **scattered free holes** into one large block; only possible with **dynamic (execution-time) relocation**.

Fragmentation	Where	Caused by
Internal	Inside allocated block	Fixed partitions / paging
External	Between blocks (scattered)	Variable partitions / segmentation

Memory Trick

- "First = fast, Best = smallest hole, Worst = largest hole." "Internal = inside, External = outside."

CBT Trap

- **First-fit** and **best-fit** beat worst-fit in storage utilization; **first-fit** is generally **fastest**. **Internal** fragmentation -> **fixed** partitioning/paging; **external** -> **variable**/segmentation. **Compaction** fixes external fragmentation, needs **dynamic relocation**.

9.7 Paging

- **Paging** divides logical memory into fixed-size **pages** and physical memory into same-size **frames**; eliminates **external** fragmentation.
- Page size = frame size = a **power of 2** (typically **4 KB**).

9.7.1 Pages, Frames, Page Table

- The **page table** maps each **page number** to a **frame number**; one entry per page; stored in memory, with base in the **PTBR (page-table base register)**.

9.7.2 Address Computation

- A logical address splits into **page number (p)** and **page offset (d)**: if logical address space = 2^m and page size = 2^n , then high-order $m-n$ bits = page number, low n bits = offset.
- **Physical address = frame number * page size + offset.**
- Number of entries in page table = $(\text{logical address space}) / (\text{page size}) = 2^{(m-n)}$.

9.7.3 TLB and EMAT

- The **TLB (Translation Lookaside Buffer)** is a small, fast **associative cache** of recent page-table entries to avoid the extra memory access.
- **Effective Memory Access Time (EMAT)** = $\text{hit_ratio} * (\text{TLB_time} + \text{mem_time}) + \text{miss_ratio} * (\text{TLB_time} + 2 * \text{mem_time})$ (for single-level page table).
- Simplified: **EMAT = $h * (c + m) + (1-h) * (c + 2m)$** , where **h** = TLB hit ratio, **c** = TLB access time, **m** = memory access time.
- Without TLB, each access needs **2** memory references (one for page table, one for data).

9.7.4 Multilevel Paging

- **Two-level paging** pages the page table itself; the outer table indexes inner tables; reduces the **contiguous** memory needed for a large page table.
- A two-level scheme needs **3** memory accesses without TLB (outer table, inner table, data).

9.7.5 Inverted Page Table

- An **inverted page table** has **one entry per physical frame** (not per page), storing **(pid, page number)**; saves memory but lookup is **slower** (search), often aided by **hashing**.

9.7.6 Internal Fragmentation in Paging

- Paging causes **internal** fragmentation only in the **last page** of a process (on average **half a page** per process).

Parameter	Formula
Page offset bits	n where page size = 2^n
Page-table entries	$2^{(m-n)}$
Physical address	frame# * page_size + offset
Accesses (no TLB, 1-level)	2
Avg internal frag	page_size / 2 per process

Memory Trick

- "Pages (logical) map to Frames (physical), same size." "No TLB = 2 memory accesses."

CBT Trap

- Paging removes **external** fragmentation but introduces **internal** fragmentation (last page). Logical address bits = **page# bits + offset bits**. **TLB** reduces access time, it is **not** where pages are stored. With a hit, only **1** memory access for data; with a miss, **2**.

9.8 Segmentation

- **Segmentation** divides a program into **variable-size** logical units (**code**, **stack**, **heap**, etc.) matching the **user's view**.
- A logical address = (**segment number s**, **offset d**); the **segment table** stores **base** and **limit** per segment.
- Physical address = **segment base + offset**, valid only if **offset < limit** (else trap).
- Allows easy **protection** and **sharing** per segment but causes **external** fragmentation.

Feature	Paging	Segmentation
Block size	Fixed (page)	Variable (segment)
User view	No (transparent)	Yes (logical units)
Fragmentation	Internal	External
Table entries	frame#	base + limit

CBT Trap

- **Paging = fixed size, internal frag**; **Segmentation = variable size, external frag**. Segmentation reflects the **programmer's logical view**; paging is **transparent** to the user.

9.9 Segmentation with Paging

- Combines both: the program is divided into **segments**, and each segment is **paged**; eliminates **external** fragmentation of pure segmentation while keeping **logical division**. Example: **Intel x86 (MULTICS-style)**.

CBT Trap

- Segmentation-with-paging removes **external** fragmentation (from segmentation) but still has small **internal** fragmentation (from paging).

PART E - Virtual Memory

10.1 Concept and Motivation

- **Virtual memory** lets a process execute even if it is **not entirely in physical memory**, giving the illusion of a very large memory.
- Benefits: programs **larger than physical memory**, higher **degree of multiprogramming**, less I/O to load programs, and easy **sharing**.
- Implemented mainly via **demand paging** (or demand segmentation).

10.2 Demand Paging; Page Fault Handling

- **Demand paging**: a page is loaded into memory **only when referenced** (lazy loading); a **valid-invalid bit** marks whether a page is in memory.
- **Page fault**: reference to a page not in memory; the steps are: (1) check reference legality, (2) find a **free frame**, (3) **schedule a disk read** of the page, (4) update the **page table** (set valid), (5) **restart** the instruction.
- A **pure demand paging** system starts a process with **zero** pages in memory.

CBT Trap

- A page fault is **not** an error - it is a normal mechanism to load pages on demand. The faulting instruction is **restarted** after the page is brought in.

10.3 Effective Access Time with Page Faults

- $EAT = (1 - p) * \text{memory_access_time} + p * \text{page_fault_service_time}$, where p = page-fault rate ($0 \leq p \leq 1$).
- Page-fault service time includes **servicing the interrupt**, **reading the page from disk** (dominant), and **restarting** the process.
- Because disk access (**~ms**) is far slower than memory (**~ns**), even a **tiny** fault rate hugely degrades performance.

Memory Trick

- " $EAT = (1-p)*ma + p*pft$." Disk dominates; keep p near zero.

CBT Trap

- In numericals, convert all times to the **same unit** (ns vs ms). A page-fault rate of even **0.001** can multiply access time many-fold. EAT formula uses **(1-p)** for hits.

10.4 Page Replacement Algorithms

- When a page fault occurs and **no free frame** exists, a **victim** page is selected and **replaced**; a **dirty/modify bit** avoids writing back unmodified pages.

10.4.1 FIFO and Belady's Anomaly

- **FIFO** replaces the **oldest** loaded page (a simple queue).
- **Belady's anomaly**: with FIFO, **increasing the number of frames can increase** the number of page faults - counterintuitive.

10.4.2 Optimal (OPT)

- **OPT (MIN)** replaces the page that **will not be used for the longest time** in the future; gives the **lowest possible** fault rate but is **unrealizable** (needs future knowledge); used as a **benchmark**.

10.4.3 LRU (Least Recently Used)

- **LRU** replaces the page **least recently used** (approximates OPT using the past); implemented with **counters** or a **stack**.
- LRU belongs to the class of **stack algorithms**, which **never** suffer Belady's anomaly.

10.4.4 Second-Chance / Clock

- **Second-chance** is FIFO with a **reference bit**: if the oldest page's reference bit is **1**, clear it and give a **second chance** (skip); the **clock** algorithm implements this with a circular list.

10.4.5 LFU / MFU

- **LFU (Least Frequently Used)**: replace the page with the **smallest reference count**.
- **MFU (Most Frequently Used)**: replace the page with the **largest count** (argument: a low-count page was just brought in).

10.4.6 Counting Faults for a Reference String

- Given a reference string and frame count, simulate the algorithm step by step; count a fault whenever a referenced page is **not in a frame**.

Algorithm	Replaces	Belady's Anomaly?	Realizable?
FIFO	Oldest loaded	Yes	Yes
OPT	Used farthest in future	No	No
LRU	Least recently used	No (stack algo)	Yes
Clock	FIFO + reference bit	Possible	Yes

Memory Trick

- "**OPT** = future, **LRU** = past." "**Stack algorithms (LRU, OPT)** never show Belady's anomaly."

CBT Trap

- **Belady's anomaly** occurs in **FIFO**, NOT in **LRU/OPT** (stack algorithms). **OPT** is the theoretical minimum but **not implementable**. Always count the **first reference** of each page as a fault in a fresh frame set.

10.5 Frame Allocation

- **Equal allocation**: divide **m** frames equally among **n** processes (**m/n** each).
- **Proportional allocation**: frames given in proportion to process **size**: $a_i = (s_i / S) * m$.
- **Global replacement**: a process can take a frame from **any** process; **local replacement**: a process replaces only its **own** frames.

CBT Trap

- **Global replacement** generally gives better throughput but makes a process's fault rate depend on **other processes**; **local** keeps frames fixed per process.

10.6 Thrashing; Locality; Working Set

- **Thrashing**: a process spends more time **paging (servicing faults)** than **executing**; happens when it has **too few frames** for its active needs; CPU utilization **collapses**.
- **Locality of reference**: processes reference a **small set of pages** intensively over a period (**temporal** and **spatial** locality).

- **Working-set model:** the **working set** $W(t, \Delta)$ is the set of pages referenced in the last **Delta** references; allocate enough frames to hold each process's working set to prevent thrashing.

CBT Trap

- **Thrashing** is caused by **too few frames / over-multiprogramming**, signaled by **high paging + low CPU use**. The fix is to **reduce the degree of multiprogramming** or use the **working-set/page-fault-frequency** scheme.

10.7 Page Size Trade-offs

- **Small page size:** less **internal** fragmentation, better resolution of locality, but **larger** page tables and more **page faults**/overhead.
- **Large page size:** smaller page tables, more efficient I/O, but more **internal** fragmentation.

CBT Trap

- Decreasing page size **increases** the page-table size and the number of faults but **decreases** internal fragmentation. There is an inherent **trade-off** - no single optimum.

PART F - File Systems

11.1 File Concept, Attributes, Types, Operations

- A **file** is a named collection of related information stored on secondary storage; the **smallest logical** unit of secondary storage from the user's view.
- Attributes:** **name**, **identifier** (unique tag/inode number), **type**, **location**, **size**, **protection**, **timestamps** (created/modified/accessed), **owner**.
- File operations:** **create**, **write**, **read**, **reposition (seek)**, **delete**, **truncate**, **open**, **close**.
- The **open-file table** keeps info on open files; **open count** tracks how many processes have a file open.

CBT Trap

- File **attributes** are stored in the **directory entry/inode**, not in the file's data. **open()** loads the directory entry into the **open-file table**; the **file descriptor** is an index into a per-process table.

11.2 Access Methods

- Sequential access:** read/write in **order**, one record after another (like tape); operations **read_next**, **write_next**.
- Direct (relative) access:** read/write any block by its **number** in **O(1)**; needed by databases.
- Indexed access:** an **index** maps keys to block locations; search the index, then access the block (like ISAM).

Method	Pattern	Random Access
Sequential	In order	No
Direct	By block number	Yes
Indexed	Via index block	Yes

CBT Trap

- Sequential** access mimics **magnetic tape**; **direct** mimics **disk**. Direct access allows immediate access to any block; sequential does not.

11.3 Directory Structures

- Single-level:** all files in **one** directory; simple but **naming conflicts** and no grouping.
- Two-level:** a separate directory **per user** (UFD) under a **master file directory (MFD)**; solves name conflicts across users.
- Tree-structured:** arbitrary **hierarchy** of directories; supports **paths** (absolute/relative); most common.
- Acyclic-graph:** allows **shared** subdirectories/files via **links** but **no cycles**; needs care with deletion (**dangling pointers**) and reference counts.
- General graph:** allows cycles; needs **garbage collection** to reclaim space.

CBT Trap

- Acyclic-graph** allows **sharing** without cycles; a **general graph** permits cycles and requires **garbage collection**. Two-level still forbids **file sharing** across users naturally.

11.4 File Protection and Access Permissions

- Protection controls who may do **read (r)**, **write (w)**, **execute (x)** on a file.
- UNIX uses **three** permission classes: **owner**, **group**, **others**, each with **r/w/x** bits -> **9** bits total, shown as e.g. **rwxr-xr--**.

- Octal representation: **r=4, w=2, x=1**; e.g., **755** = rwx for owner, r-x for group/others; **644** = rw-r--r--.
- The **chmod** command sets permissions; **chown** changes owner; **umask** sets default permission mask.

Permission	Octal	Bit
read	4	r
write	2	w
execute	1	x

Memory Trick

- "**r=4, w=2, x=1**; add them. 755 = rwxr-xr-x, 644 = rw-r--r--."

CBT Trap

- **r=4, w=2, x=1**. **chmod 777** = full permission for all (insecure). For a **directory**, **x** permits **entering/traversing**, not executing.

11.5 File Allocation Methods

11.5.1 Contiguous Allocation

- Each file occupies a set of **contiguous** blocks; needs only **start block + length** in the directory.
- Supports both **sequential and direct** access efficiently; suffers **external fragmentation** and difficulty in **growing** files.

11.5.2 Linked Allocation (and FAT)

- Each file is a **linked list** of disk blocks scattered anywhere; directory holds **first** (and last) block pointers; each block stores a **pointer** to the next.
- No external fragmentation, files can grow; but **direct access is poor** (must traverse) and pointers consume space; a **single corrupted pointer** breaks the chain.
- **FAT (File Allocation Table)**: places all block pointers in a **table at the start** of the volume, improving {{random access}} (used by MS-DOS, FAT12/16/32).

11.5.3 Indexed Allocation

- Each file has an **index block** holding **all** its block pointers; supports **direct access** without external fragmentation; overhead is the **index block** space; max file size limited by index block size.

11.5.4 UNIX inode

- A UNIX **inode** holds file attributes plus pointers: typically **12 direct** block pointers, **1 single**-indirect, **1 double**-indirect, and **1 triple**-indirect pointer.
- Direct pointers reach small files quickly; indirect pointers extend the maximum size for large files.

11.5.5 Max File Size Computation

- Let block size = **B** bytes and each pointer = **P** bytes, so a block holds **k = B/P** pointers.
- Max file size = **(12 + k + k^2 + k^3) * B** bytes (with 12 direct, single, double, triple indirect).
- Single indirect adds **k * B**; double indirect adds **k^2 * B**; triple indirect adds **k^3 * B**.

Allocation	Direct Access	Fragmentation	Grows Easily
Contiguous	Excellent	External	No
Linked	Poor	None (external)	Yes
Indexed	Good	None (external)	Yes

Memory Trick

- "inode: **12 direct + single + double + triple.**" Max = $(12 + k + k^2 + k^3) * B$, $k = B/P$.

CBT Trap

- **Contiguous** = external fragmentation but best access; **linked** = no direct access; **indexed/inode** = best general purpose. In inode numericals compute **k = block/pointer size** first, then sum **12 + k + k^2 + k^3**.

11.6 Free-Space Management

- **Bit vector / bitmap**: one **bit per block** (**0 = free, 1 = used** or vice-versa); easy to find contiguous runs but needs the bitmap in memory.
- **Linked list**: chain all **free** blocks; no waste but **traversal** is slow and no easy contiguous allocation.
- **Grouping**: the first free block stores addresses of **n** free blocks (the last points to the next group); faster than linked list.
- **Counting**: store **address + count** of consecutive free blocks; good when free space is **clustered**.

CBT Trap

- **Bitmap** makes finding the **first free** block easy but needs space proportional to disk size; **counting** works best when free blocks are **contiguous/clustered**.

11.7 Directory Implementation

- **Linear list**: directory is a list of (name, pointer); simple but search is **O(n)**.
- **Hash table**: hashes file name to an entry; search is **O(1)** average but suffers **collisions** and fixed table size.

CBT Trap

- **Linear list** directory search = **O(n)**; **hash table** = **O(1)** average but must handle **collisions**.

PART F - Disk / Secondary Storage

12.1 Disk Structure

- A **hard disk** has **platters**, each surface divided into concentric **tracks**; tracks are divided into **sectors**; the set of same-position tracks across platters is a **cylinder**.
- The **read/write head** moves over the surface; data location = (cylinder, head/surface, sector) = **CHS**.

CBT Trap

- A **cylinder** = all tracks at the same radius across **all** platters; a **sector** is the smallest addressable storage unit. Moving heads = **seek**; spinning platter = **rotation**.

12.2 Disk Performance

- **Seek time**: time to move the head to the correct **cylinder/track** (usually the **dominant** component).
- **Rotational latency**: time for the desired **sector** to rotate under the head; on average **half** a rotation.
- **Transfer time**: time to actually transfer the data.
- **Disk access time = seek time + rotational latency + transfer time**.
- Average rotational latency = $(1/2) * (60 / \text{RPM})$ seconds = $30/\text{RPM} * 1000$ ms.

Memory Trick

- "**Access = Seek + Latency + Transfer**." Average latency = **half rotation**.

CBT Trap

- Average rotational latency = time for **half** a revolution = $0.5 * (60/\text{RPM})$ s. **Seek** time usually dominates. Convert **RPM** to ms carefully (e.g., 7200 RPM -> one rotation = **8.33 ms**, latency ~**4.17 ms**).

12.3 Disk Scheduling Algorithms

- Goal: minimize **total head movement (seek)** to improve throughput.

12.3.1 FCFS

- Serve requests in **arrival order**; fair but can cause large, wasteful head movement.

12.3.2 SSTF (Shortest Seek Time First)

- Serve the request **closest** to the current head position; reduces movement but can **starve** far requests; analogous to **SJF**.

12.3.3 SCAN (Elevator)

- The head moves in one **direction** servicing requests until the **end** of the disk, then **reverses**; like an elevator.

12.3.4 C-SCAN (Circular SCAN)

- The head moves in one direction to the **end**, then **jumps back** to the start **without servicing** on the return, giving more **uniform** wait times.

12.3.5 LOOK

- Like SCAN but the head only goes as far as the **last request** in each direction (does not go to the physical end).

12.3.6 C-LOOK

- Like C-SCAN but reverses at the **last request** rather than the disk end, then jumps to the **first** request.

12.3.7 Total Head-Movement Computation

- Sum the **absolute differences** of consecutive head positions along the service order; this total = **total head movement**.

Algorithm	Idea	Starvation	Like
FCFS	Arrival order	No	FIFO
SSTF	Nearest first	Yes	SJF
SCAN	Sweep to end, reverse	Low	Elevator
C-SCAN	Sweep, jump to start	Low	Circular
LOOK	Sweep to last request	Low	SCAN variant
C-LOOK	Sweep to last, jump	Low	C-SCAN variant

Memory Trick

- "SSTF = SJF of disks (can starve)." "SCAN goes to the wall; LOOK only to the last request."

CBT Trap

- SCAN/C-SCAN** go to the **physical end**; **LOOK/C-LOOK** stop at the **last request**. **C-SCAN/C-LOOK** give **uniform** wait by not servicing on the return sweep. Always note the **initial head position** and **direction** - both change the answer.

12.4 RAID Levels 0-5

- RAID (Redundant Array of Independent Disks)** combines multiple disks for **performance** and/or **reliability**.
- RAID 0: striping**, no redundancy; best **performance**, **no** fault tolerance.
- RAID 1: mirroring**; full duplicate; high reliability, **50%** space overhead.
- RAID 2: bit-level striping with Hamming code ECC** (rarely used).
- RAID 3: byte-level striping with a dedicated parity disk**.
- RAID 4: block-level striping with a dedicated parity disk**.
- RAID 5: block-level striping with distributed parity**; survives **one** disk failure; popular.
- RAID 6: like 5 but double distributed parity**; survives **two** disk failures (extension).

RAID	Technique	Fault Tolerance
0	Striping	None
1	Mirroring	1 disk (mirror)
3	Byte stripe + dedicated parity	1 disk
4	Block stripe + dedicated parity	1 disk
5	Block stripe + distributed parity	1 disk

CBT Trap

- RAID 0** = striping only (no redundancy, fastest); **RAID 1** = mirroring; **RAID 5** = **distributed** parity (vs RAID 4's dedicated parity). RAID 0 provides **no** fault tolerance - a favorite trap.

PART G - I/O Systems

13.1 I/O Hardware

- I/O devices connect via **ports**, **buses**, and **device controllers**; a controller has **registers** (**data-in**, **data-out**, **status**, **control**).
- Memory-mapped I/O**: device registers mapped into the **address space**; **port-mapped I/O**: special **IN/OUT** instructions.

13.2 Programmed I/O (Polling)

- Programmed I/O**: the CPU **busy-waits**, repeatedly checking the **status (busy/ready) bit** (polling) until the device is ready, then transfers data.
- Simple but **wastes CPU** cycles - the CPU does **nothing useful** while polling.

13.3 Interrupt-Driven I/O; ISR

- The device **interrupts** the CPU when ready; the CPU saves state and runs an **Interrupt Service Routine (ISR)** via the **interrupt vector**, then resumes.
- Avoids busy waiting; the CPU does **other work** while the device operates.
- Maskable** interrupts can be deferred; **non-maskable (NMI)** cannot (critical errors).

CBT Trap

- Polling** wastes CPU (busy wait); **interrupt-driven** I/O frees the CPU. The **interrupt vector** maps interrupt numbers to ISR addresses. NMI cannot be **masked**.

13.4 Direct Memory Access (DMA)

- DMA** lets a **DMA controller** transfer data **directly between device and memory** without the CPU moving each byte; the CPU is involved only at **start** and **end (interrupt)**.
- Used for **high-speed, bulk** transfers (disk, network); reduces CPU overhead drastically.
- Cycle stealing**: the DMA controller **steals** bus cycles from the CPU to transfer data.

CBT Trap

- DMA interrupts the CPU **once per block** (at completion), not per byte/word - that is its key advantage over interrupt-driven I/O. **Cycle stealing** is the term for DMA borrowing the bus.

13.5 Buffering, Caching, Spooling

- Buffering**: a memory area that **holds data** during transfer to cope with **speed mismatch** between producer and consumer.
- Caching**: keeping **copies** of frequently used data in faster storage for quick reuse.
- Spooling (Simultaneous Peripheral Operation On-Line)**: buffering output for a **non-sharable** device (e.g., **printer**) on disk so jobs queue up; classic **print spooler**.

Technique	Purpose
Buffering	Smooth speed mismatch
Caching	Reuse hot data (faster copy)
Spooling	Queue output for printer

CBT Trap

- A **cache** holds a **copy** of data that exists elsewhere; a **buffer** may hold the **only** copy in transit. **Spooling** is most associated with the **printer**.

PART G - Protection and Security

14.1 Goals of Protection; Access Matrix

- **Protection** controls access of **processes/users** to system resources; enforces the **principle of least privilege**.
- The **access matrix** is an abstract model: **rows = domains**, **columns = objects**, each cell lists **allowed operations**.

14.2 Access Control Lists vs Capabilities

- **Access Control List (ACL)**: store the matrix by **column** - each object lists which domains may access it and how.
- **Capability list**: store the matrix by **row** - each domain holds a list of objects (capabilities/tickets) it may access.

Model	Stored by	Question answered
ACL	Column (per object)	Who can access this object?
Capability	Row (per domain)	What can this domain access?

CBT Trap

- **ACL** = per-**object** list (column-wise); **Capability** = per-**domain/user** list (row-wise). They are **two decompositions** of the same access matrix.

14.3 Authentication Basics

- **Authentication** verifies identity using **something you know** (password), **something you have** (token/smart card), or **something you are** (biometrics).
- **Passwords** should be **salted and hashed**; **multi-factor authentication (MFA)** combines two or more categories.

CBT Trap

- The **three authentication factors** are **knowledge (know)**, **possession (have)**, and **inherence (are)**. Passwords stored as **salted hashes**, never plaintext.

PART H - CIL MT Exam Focus and Numerical Toolkit

15.1 CPU Scheduling Formulas (Recap)

- **Turnaround Time (TAT) = Completion Time - Arrival Time.**
- **Waiting Time (WT) = Turnaround Time - Burst Time.**
- **Response Time = First CPU allocation time - Arrival Time.**
- **Average WT = (sum of all WT) / n; Average TAT = (sum of all TAT) / n.**
- **SJF** minimizes average waiting time; **RR** -> FCFS as quantum grows.

15.2 Banker's Algorithm (Recap)

- **Need = Max - Allocation**; a **safe sequence** exists iff the system is in a **safe state**; safe => **no deadlock**.
- Grant a request only if the resulting state remains **safe** (run the safety algorithm).

15.3 Page Replacement (Recap)

- **FIFO** can show **Belady's anomaly**; **LRU/OPT** (stack algorithms) cannot.
- **OPT** gives minimum faults but is **unimplementable**.

15.4 EMAT / TLB / Paging (Recap)

- **EMAT = $h \cdot (c+m) + (1-h) \cdot (c+2m)$** (single-level page table, c = TLB time, m = memory time, h = hit ratio).
- Without TLB, single-level paging needs **2** memory accesses; two-level needs **3**.
- **EAT (demand paging) = $(1-p) \cdot m_a + p \cdot \text{page_fault_time}$.**

15.5 Address Translation (Recap)

- Page size = **2^n** -> offset = **n** bits; page-table entries = **logical size / page size**.
- **Physical address = frame number * page size + offset.**
- Segmentation: physical = **segment base + offset**, valid if **offset < limit**.

15.6 Disk Scheduling (Recap)

- **Disk access = seek + rotational latency + transfer**; avg latency = **half a rotation**.
- Total head movement = sum of **|consecutive position differences|** in service order.

15.7 Semaphore Reasoning (Recap)

- Bounded buffer: **mutex=1, empty=n, full=0**; acquire resource semaphore **before** mutex.
- **wait = P = down = decrement**; **signal = V = up = increment**.

15.8 inode Max File Size (Recap)

- **$k = \text{block size} / \text{pointer size}$** ; Max file size = **$(12 + k + k^2 + k^3) * \text{block_size}$.**

15.9 fork() Process Count (Recap)

- **n** forks -> **2^n** total processes, **$2^n - 1$** children.

15.10 Definitions and One-Liner Anchors

- **5** process states: **new, ready, running, waiting, terminated**.
- **2** fragmentation types: **internal** (fixed/paging) and **external** (variable/segmentation).
- **4** deadlock conditions: **mutual exclusion, hold-and-wait, no preemption, circular wait**.
- **3** CS requirements: **mutual exclusion, progress, bounded waiting**.

CBT Trap

- Keep units consistent in all numericals; re-read whether priority convention is **low number = high priority**; watch arrival times in scheduling; and always recompute **Need = Max - Allocation** before the safety check.

Final Rapid-Revision Comparison Table

Concept	Key Number / Fact
Process states	5
Deadlock conditions	4
CS requirements	3
inode pointer classes	12 direct + 3 indirect
Avg rotational latency	half rotation
Belady's anomaly	FIFO only
Fastest IPC	Shared memory
Min avg waiting time	SJF
Uncatchable signals	SIGKILL(9), SIGSTOP
Mode bit	0=kernel, 1=user